МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

Дисциплина:

Операционные системы и архитектура компьютера

Тема:

Межпроцессное взаимодействие с использованием разделяемой памяти

Вариант 19

Выполнил:
студент группы М80-203БВ-25
Прудников Я. А.

Преподаватель:
Миронов Е. С.

Москва, 2026 г.

1 Цель работы

Изучить создание процессов и обмен данными между ними через отображаемую разделяемую память. Разработать программу на языке C, в которой родительский процесс распределяет строки между двумя дочерними процессами, а дочерние процессы обрабатывают строки и записывают результат в файлы.

2 Постановка задачи

Вариант 19 сохраняет условие лабораторной работы № 2: родительский процесс получает два имени выходных файлов, создаёт два дочерних процесса и передаёт каждому из них строки. Каждая строка с вероятностью 80% направляется первому процессу и в остальных случаях – второму. Дочерние процессы удаляют из строк гласные и записывают результат в соответствующий файл. В работе № 4 вместо каналов используется отображаемая память; системные ошибки необходимо обрабатывать.

3 Архитектура решения

В проекте родительская и дочерняя программы находятся в файлах `parent.c` и `child.c`. Общие функции чтения и обработки строк вынесены в `line_utils.c`, а интерфейс работы с процессами и памятью – в `os_api.h`. Реализация для Linux находится в `platform/os_linux.c`. Исходный код проекта: `https://github.com/yalama34/os-lab-3`.

Родитель считывает имена файлов, проверяет, что они непустые и различаются, затем создаёт два объекта POSIX shared memory. Для каждого объекта выделяется собственный отображённый буфер. Через `fork()` создаётся процесс, в нём стандартный вывод перенаправляется в файл вызовом `dup2()`, после чего запускается программа `child` через `execvp()`. Имя объекта памяти передаётся ей аргументом. Родитель читает строки до конца ввода и выбирает адресата с помощью `rand() % 100`: значения от 0 до 79 соответствуют первому процессу.

Дочерний процесс открывает объект памяти, собирает переданную строку и удаляет символы `a, e, i, o, u, y` и их прописные варианты. Затем он записывает результат в стандартный вывод, который уже направлен в файл. Обработка кириллических гласных исходным кодом не предусмотрена.

3.1 Схема взаимодействия компонентов

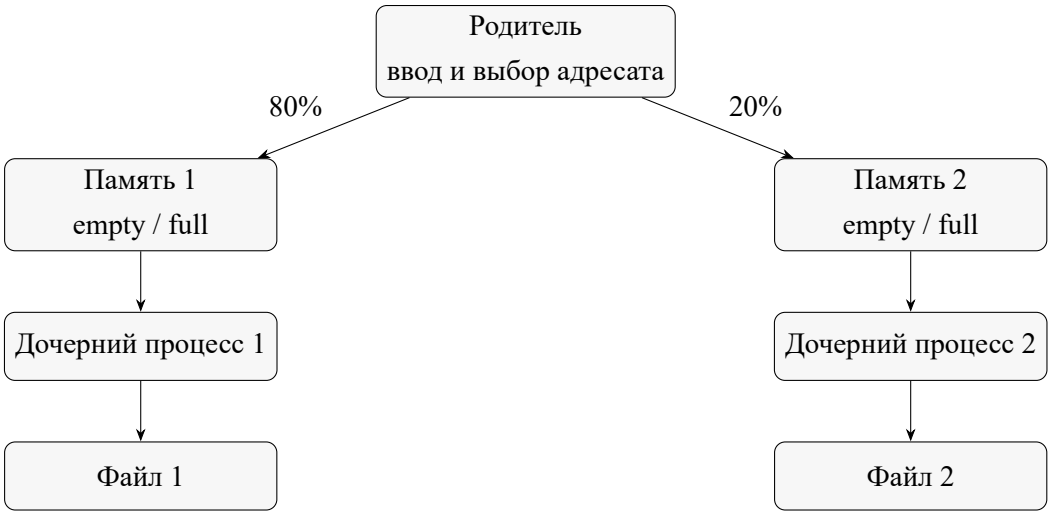


Рис. 1: Обмен строками между процессами

3.2 Организация обмена

Объект памяти создаётся через `shm_open()` и `ftruncate()`, затем отображается вызовом `mmap()` с флагом `MAP_SHARED`. В каждом буфере есть массив данных на 4096 байт, длина фрагмента, признаки последнего фрагмента и завершения передачи, а также два межпроцессных семафора. Семафор `empty` сначала равен единице, `full` – нулю. Записывающий процесс ждёт освобождения буфера, помещает фрагмент и сигнализирует о готовности; читающий ждёт данные, копирует их и освобождает буфер. Строки длиннее 4096 байт передаются по частям и собираются в дочернем процессе.

После конца ввода родитель отправляет каждому процессу сообщение с признаком завершения и вызывает `waitpid()`. Затем отображения и дескрипторы закрываются, а владелец удаляет объекты памяти и семафоры. В коде проверяются результаты основных системных вызовов; ожидание семафоров и процессов повторяется при прерывании сигналом `EINTR`.

4 Ключевые фрагменты кода

Ниже приведены фрагменты из исходных файлов проекта. Полные файлы доступны в репозитории.

4.1 Создание отображения и семафоров

Объект общей памяти получает размер структуры буфера, после чего в нём создаются семафоры, доступные разным процессам.

```
new_memory->fd = shm_open(name, O_CREAT | O_EXCL | O_RDWR, 0600);
ftruncate(new_memory->fd, sizeof(shared_buffer));
void *mapping = mmap(NULL, sizeof(shared_buffer),
    PROT_READ | PROT_WRITE, MAP_SHARED, new_memory->fd, 0);
new_memory->buffer = mapping;
sem_init(&new_memory->buffer->empty, 1, 1);
sem_init(&new_memory->buffer->full, 1, 0);
```

Листинг 1 – Инициализация общей памяти (проверки ошибок опущены)

4.2 Передача фрагмента строки

Отправитель дожидается свободного буфера и сообщает получателю о появлении данных. Признак `is_last` позволяет получателю определить конец строки.

```
size_t part = length - offset;
if (part > BUFFER_SIZE) {
    part = BUFFER_SIZE;
}
if (wait_semaphore(&buffer->empty) == -1) {
    perror("sem_wait(empty)");
    return -1;
}
buffer->length = part;
buffer->is_last = (offset + part == length);
buffer->finished = 0;
if (part > 0) {
    memcpy(buffer->data, data + offset, part);
}
if (sem_post(&buffer->full) == -1) {
    perror("sem_post(full)");
    return -1;
}
offset += part;
```

Листинг 2 – Запись одного фрагмента строки

4.3 Распределение и обработка строк

В родительском процессе выбор получателя выполняется для каждой прочитанной строки.

```
int choice = rand() % 100;
if (choice < 80) {
    os_shm_write(memory1, line, length);
} else {
    os_shm_write(memory2, line, length);
}
```

Листинг 3 – Выбор дочернего процесса (проверки ошибок опущены)

Дочерняя программа читает строку из своего объекта памяти, удаляет гласные функцией `remove_vowels()` и пишет оставшиеся символы в файл через стандартный вывод.

```
int result = os_shm_read(memory, &line, &length);
remove_vowels(line);
if (fputs(line, stdout) == EOF) {
    free(line);
    os_shm_destroy(memory);
    return 1;
}
free(line);
```

Листинг 4 – Обработка строки дочерним процессом

5 Состояние реализации и проверка

При анализе исходных файлов обнаружены несогласованные объявления интерфейса: в `os_api.h` третий аргумент `os_shm_read` объявлен как `size_t`, тогда как реализация и вызов используют `size_t *`; второй аргумент `os_shm_write` объявлен как `const void *`, а в определении – как `const char *`. Кроме того, заголовок объявляет `os_destroy_shm`, тогда как реализация и вызывающий код используют `os_shm_destroy`. Эти расхождения необходимо исправить перед сборкой.

Проверка запуска и получение журналов процессов не выполнены: доступная среда WSL вернула системную ошибку при запуске. Поэтому фактический консольный вывод, идентификаторы процессов и содержимое результирующих файлов в отчёте не приводятся. В проекте также нет сценария сборки или сохранённых результатов тестов. Распределение 80/20 описывает правило выбора для отдельных строк; точное соотношение в конечной выборке не гарантируется. При аварийном завершении дочернего процесса ожидание семафора родителем может остаться заблокированным.

6 Выводы

В проекте подготовлена реализация обмена строками между родительским и двумя дочерними процессами через отображаемую POSIX-память. Передача строк разделена на фрагменты, доступ к буферам синхронизируется семафорами, а результат дочерних процессов направляется в отдельные файлы. В ходе работы изучены принципы отображения памяти с помощью вызова `mmap()`, синхронизации процессов семафорами и передачи данных через общий буфер.